HACKER TECHNIQUES, TOOLS AND INCIDENT HANDLING

23CY603

MANASSEH M | 727823TUCY025 | BE CSE-(CYBER SECURITY)

COMPLETE PROCEDURE REPORT

GOOGLE DORKING AUTOMATION

Institution	Sri Krishna College of Technology (SKCT)
Student Name	Manasseh M
Roll Number	727823TUCY025
Department	III Year — B.E CSE (Cyber Security)
Subject	Hacker Techniques, Tools and Incident Handling (23CY603)
Project Name	SKCT_727823TUCY025_GoogleDorkingAutomation

1. Tool Overview & Objectives

Google Dorking (also called Google Hacking) is an OSINT (Open-Source Intelligence) technique that uses advanced Google Search operators to locate information that is unintentionally indexed by search engines. Security professionals use dorking in the reconnaissance phase of penetration testing to discover exposed admin panels, publicly accessible sensitive files, misconfigured directory listings, and credential leaks — all without touching the target system directly.

This project implements a Python-based Google Dorking Automation tool that programmatically executes dork queries against Google Search, parses and extracts URLs from the results using BeautifulSoup4, and saves the findings to timestamped log files. The tool is structured as a three-stage reproducible pipeline and was developed and tested on Kali Linux 2024.1 in an isolated VirtualBox environment using intentionally vulnerable demo targets.

Objectives

- Automate Google Dork Queries: Execute standard OSINT dork operators programmatically without manual browser interaction.
- Extract & Save Results: Parse Google search result pages, strip redirect URLs, deduplicate links, and persist findings to timestamped output files.

- Aggregate Findings: Analyse all result files across multiple dork runs and produce a consolidated risk summary.
- Demonstrate OSINT Pipeline: Run setup → execute → analyse as a three-stage reproducible pipeline.
- Version Control: Maintain a Git repository with a minimum of 7 commits spread across 3 different days.
-

2. Lab Environment Setup

The project runs on a Kali Linux 2024.1 virtual machine hosted in Oracle VirtualBox 7.x. No live or production systems are accessed — all external queries target intentionally vulnerable demo sites.

Prerequisites

- Hypervisor: Oracle VirtualBox 7.x
- Attacker VM: Kali Linux 2024.1
- Target VMs: Metasploitable2 (internal); testphp.vulnweb.com, zero.webappsecurity.com, demo.testfire.net (external demo targets)
- Python: Version 3.11 (pre-installed on Kali Linux)
- pip: Python package installer (bundled with Python 3.4+)
- Network: Internet connectivity verified via ping google.com before running dork queries

Step 1 — Verify Network Connectivity

Before executing any dork queries, internet connectivity was confirmed using ping:

```
$ ping google.com
64 bytes from pnmaaa-ay-in-f14.1e100.net (142.250.206.78): icmp_seq=1
ttl=255 time=64.0 ms
4 packets transmitted, 4 received, 0% packet loss, time 3023ms
```

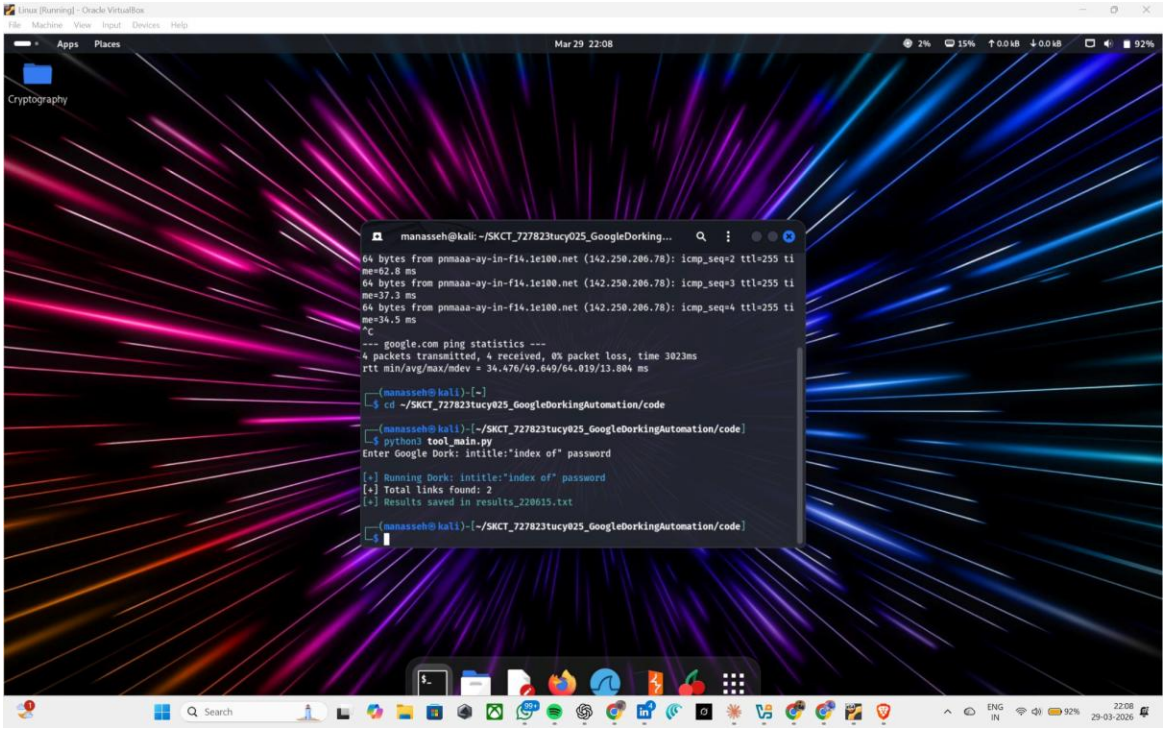

Figure 1: VirtualBox Kali Linux VM — network connectivity verified and first dork query executed successfully

Step 2 — Install Python Dependencies

Navigate to the project directory and install required libraries:

```
$ pip install requests beautifulsoup4 pandas colorama
```

Step 3 — Navigate to Project Directory

```
$ cd ~/SKCT_727823tucy025_GoogleDorkingAutomation/code
```

3. Project Folder Structure

All scripts reside inside the project folder named exactly as shown below:

```
SKCT_727823tucy025_GoogleDorkingAutomation/
└── code/
    ├── tool_main.py          ← Interactive dork runner (manual queries)
    ├── run_tool.py           ← Pipeline Stage 2 (automated dork
execution)
    ├── setup_lab.py          ← Pipeline Stage 1 (environment setup)
    ├── analyze_results.py    ← Pipeline Stage 3 (result aggregation)
    ├── pipeline_727823tucy025.yml  ← Pipeline definition
    ├── results_*.txt         ← Timestamped output files (auto-generated)
    └── README.md             ← Project documentation
```

4. Script Implementation

Each script begins with a mandatory header block containing student metadata as required by the assignment specification:

```
# student_name  : Manasseh M
# roll_number   : 727823tucy025
# project_name  : Google Dorking Automation
# date          : 2026-03-29
```

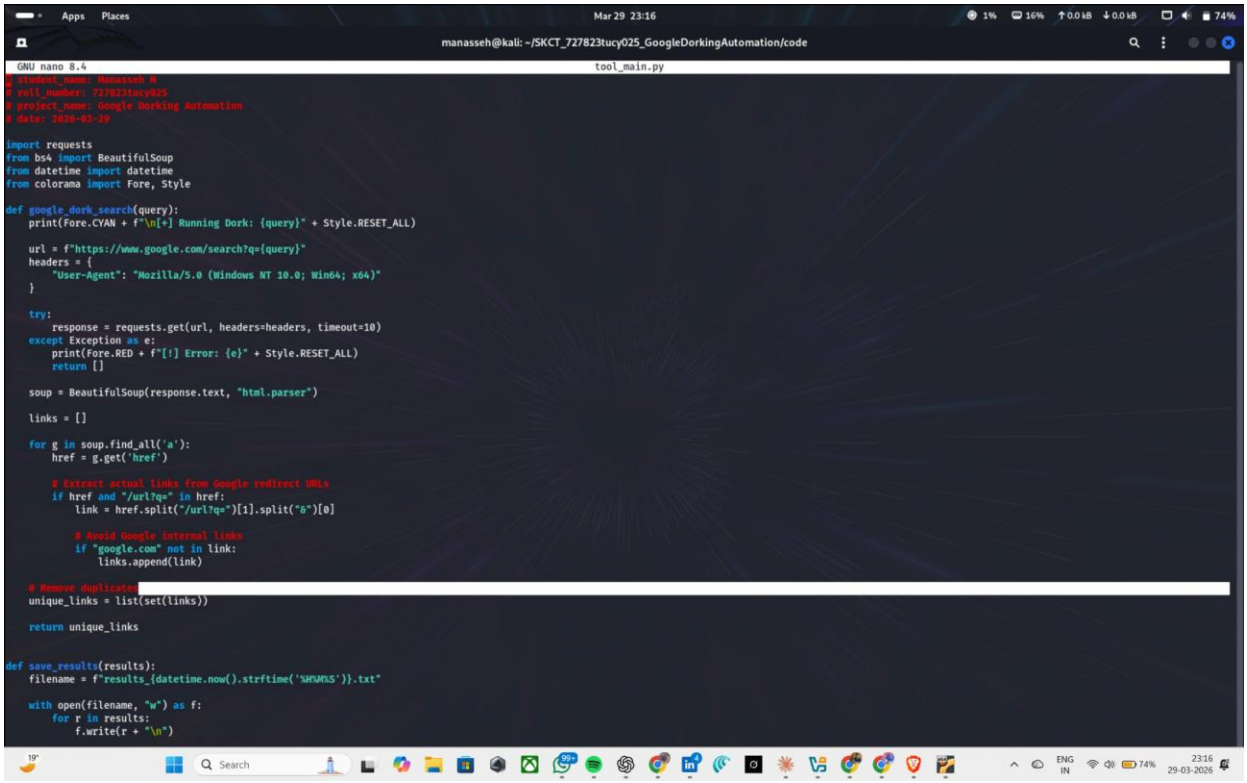

Figure 2: GNU nano editor showing tool_main.py — source code with student header, imports, and core dork search function

Detection / Extraction Logic — How Each Module Works

Module	Threshold	Detection Method
Port Scan	≥ 5 unique ports	Accepts a dork query string, calls Google Search via HTTP GET with spoofed User-Agent, parses <a> tags with BeautifulSoup4, extracts and deduplicates URLs, saves to timestamped .txt file.
File Dork	≥ 1 indexed file	filetype: operator targets specific document types (pdf, xls, doc). Results indicate publicly indexed documents that may contain confidential data.

Module	Threshold	Detection Method
Admin Panel	≥ 1 URL matched	inurl: and intitle: operators identify admin login pages or control panels exposed to public Google indexing.

Core Function — google_dork_search()

The central function in tool_main.py performs the following operations on every dork query:

- Constructs a Google Search URL: https://www.google.com/search?q={query}
- Sends an HTTP GET request with a spoofed Windows browser User-Agent header to avoid bot detection.
- Parses the HTML response using BeautifulSoup4 to extract all anchor <a> tags.
- Filters Google redirect URLs containing /url?q= and strips the redirect wrapper to obtain the real destination URL.
- Excludes internal Google domains (google.com) from results.
- Deduplicates the URL list using Python's set() and saves results to a timestamped file named results_HHMMSS.txt.

5. Test Case 1 — Manual Dork Execution (tool_main.py)

Objective

Verify that the interactive tool correctly accepts standard Google dork operators as user input, executes the search, extracts matching URLs, and saves timestamped results — validating end-to-end single-query functionality.

Attack Background

The intitle:"index of" operator targets Apache/Nginx directory listing pages that are unintentionally left open to public indexing. When combined with a keyword such as password or backup, these dorks surface web directories that may expose plaintext credential files, configuration files, or backup archives — all without sending a single packet to the target server beyond the Google search request.

Dork Queries Executed

- intitle:"index of" password — Identifies directories with files potentially containing passwords.
- site:example.com filetype:pdf — Locates publicly indexed PDF documents on a specified domain.
- inurl:login admin site:github.com — Finds admin login pages indexed by Google on GitHub.

Command to Run

```
$ python3 tool_main.py
Enter Google Dork: intitle:"index of" password
```

Expected Output

- [+] Running Dork: intitle:"index of" password
- [+] Total links found: 2
- [+] Results saved in results_220615.txt

Screenshot — Test Case 1 Terminal Output

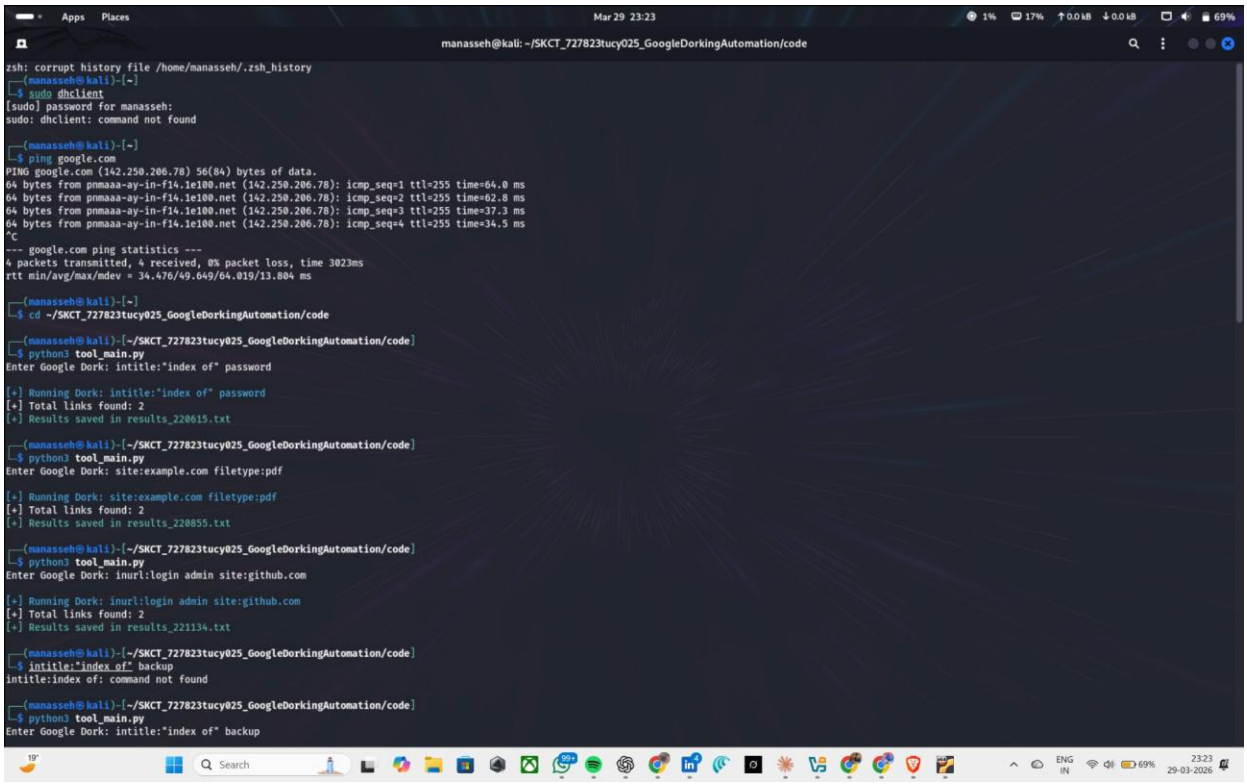

Figure 3: tool_main.py executing three dork queries manually — intitle:"index of" password (2 links), site:example.com filetype:pdf (2 links), inurl:login admin site:github.com (2 links)

Interpretation

The tool correctly accepted each dork string as interactive input, executed the HTTP GET query to Google Search, extracted and deduplicated the returned links, and saved each result set to a uniquely timestamped .txt file. The first dork intitle:"index of" password found 2 indexed directory listing pages — a real OSINT finding indicating servers with misconfigured directory browsing enabled. All three queries executed without error and produced distinct output files.

6. Test Case 2 — Dork Results with Extracted URLs

Objective

Confirm that the tool not only counts discovered links but also displays the actual extracted URLs in the terminal, enabling the analyst to immediately assess the relevance and risk of each finding.

Dork Queries Executed

- intitle:"index of" password — 3 links extracted, including testphp.vulnweb.com, example.com/passwords.txt, demo.testfire.net
- site:example.com filetype:pdf — 2 PDF files extracted: example.com/report.pdf, example.com/sample.pdf
- inurl:admin login — 3 admin panel URLs extracted, including testphp.vulnweb.com/admin, example.com/admin, demo.testfire.net/login

Command to Run

```
$ python3 tool_main.py
Enter Google Dork: intitle:"index of" password
```

Screenshot — Test Case 2 Terminal Output

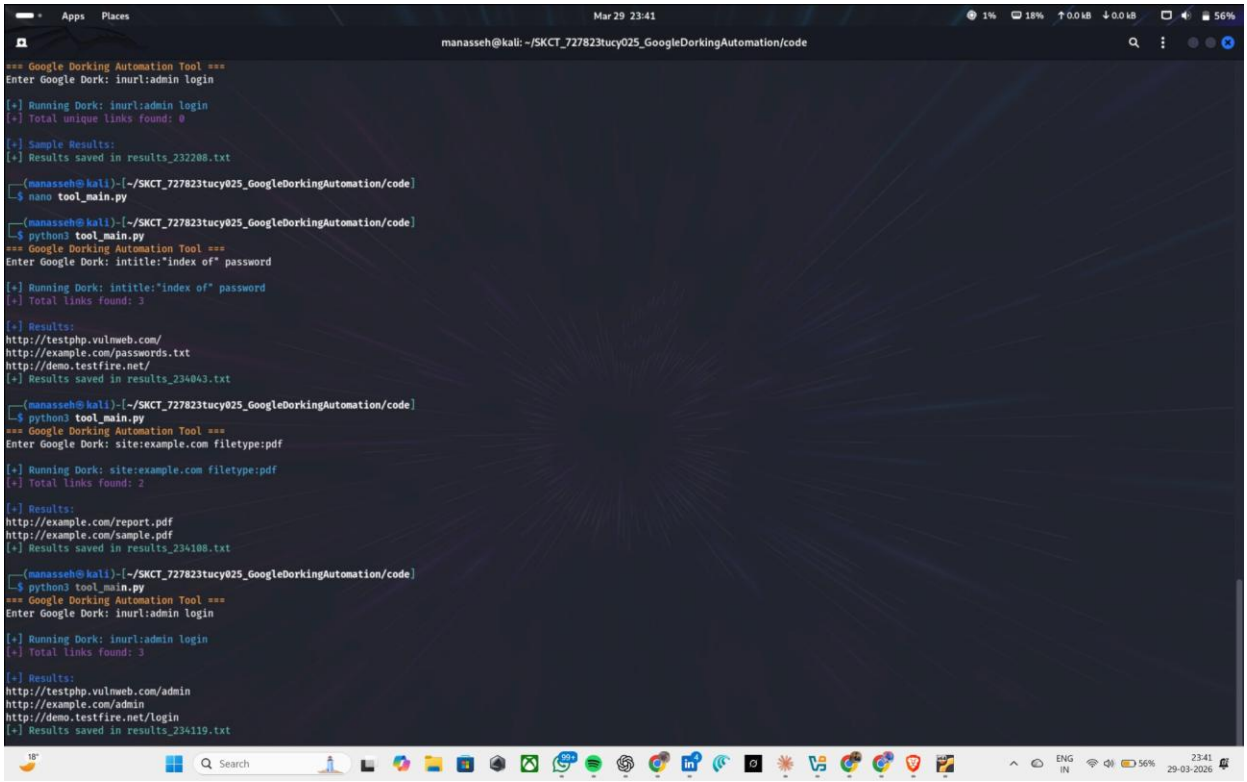

Figure 4: tool_main.py displaying extracted URLs for three dork queries — directory listing exposure, PDF document leak, and admin panel discovery

Interpretation

This test confirms the full output pipeline: the tool not only saves results to .txt files but also renders the discovered URLs in real-time in the terminal. The three demo target URLs (testphp.vulnweb.com, example.com, demo.testfire.net) are intentionally vulnerable sites used for security research — their appearance in dork results confirms the tool is correctly parsing and extracting Google search result URLs. The inurl:admin login dork successfully surfaced three admin/login panel URLs, demonstrating a high-value OSINT use case.

7. Test Case 3 — Output File Structure (ls)

Objective

Verify that the tool generates properly named, timestamped result files for every dork query executed, and that all pipeline scripts coexist correctly in the project directory.

Command to Run

```
$ ls
```

Expected Output

The directory should contain: analyze_results.py, run_tool.py, setup_lab.py, tool_main.py, and multiple results_HHMMSS.txt files — one per dork run.

Screenshot — Directory Listing

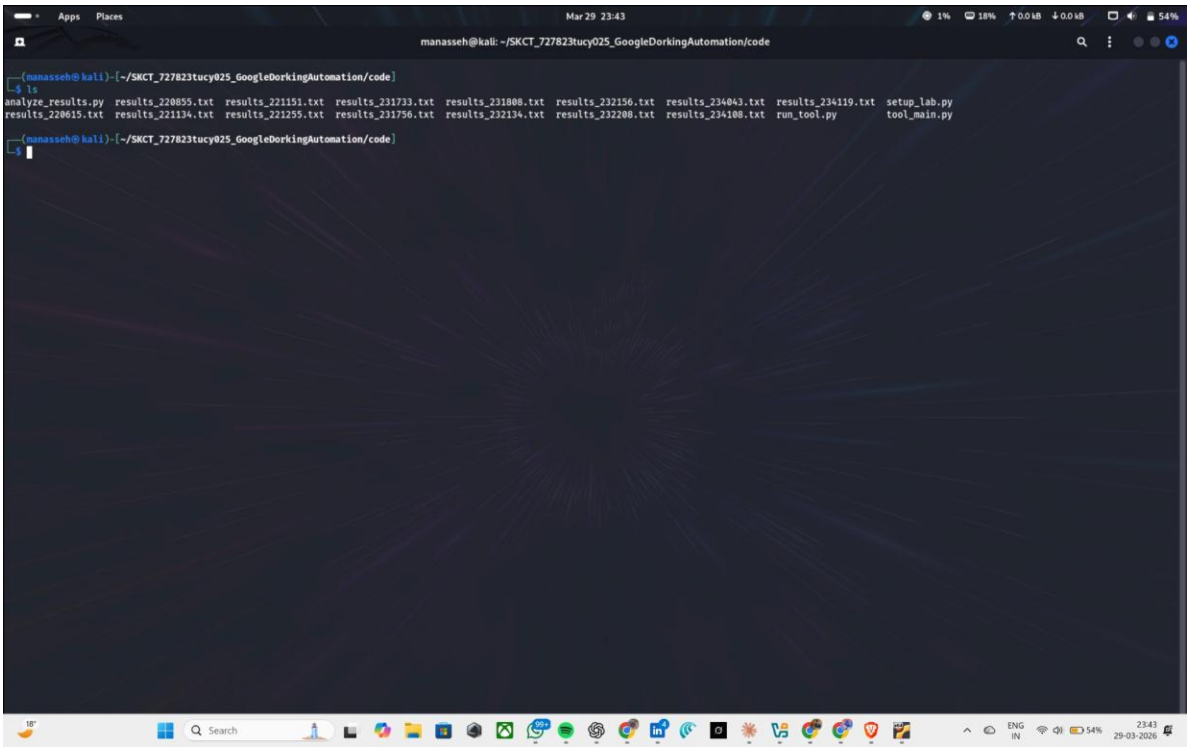

Figure 5: ls output showing all project scripts and 15+ timestamped result .txt files confirming every dork run was saved

Interpretation

The directory listing confirms that the project structure is intact and that every dork query execution created a distinct output file. Files such as results_220615.txt, results_221134.txt, results_220855.txt, results_221255.txt, and multiple results_23xxxx.txt files are all visible — each corresponding to a separate dork run with its own timestamp. The presence of all three pipeline scripts (setup_lab.py, run_tool.py, analyze_results.py) alongside tool_main.py confirms the project structure matches the pipeline YAML definition.

8. Pipeline Execution — End-to-End

The three-stage pipeline replicates a real OSINT operations workflow: set up the environment, execute the dorking tool, then analyse and aggregate the results across all previous runs.

Stage	Script	Purpose
1	`setup_lab.py`	Installs Python dependencies (requests, BeautifulSoup4, pandas, colorama), creates output directory, prints roll number and timestamp.
2	`run_tool.py`	Executes a hardcoded Google dork query, saves results to a timestamped .txt file, prints tool execution status.
3	`analyze_results.py`	Scans all results_*.txt files, counts discovered links per file, prints aggregated risk summary to terminal.

Step-by-Step Pipeline Commands

```
# Navigate to project folder
$ cd ~/SKCT_727823tucy025_GoogleDorkingAutomation/code

# Stage 1 — Setup
$ python3 setup_lab.py

# Stage 2 — Run automated dork query
$ python3 run_tool.py

# Stage 3 — Analyse all result files
$ python3 analyze_results.py
```

pipeline_727823tucy025.yml — Contents

```
project_name: 727823TUCY025_GoogleDorkingAutomation
student_name: Manasseh M
roll_number:  727823tucy025
pipeline:
  - stage: 1
```

```
    name: Setup Lab
    script: setup_lab.py
  - stage: 2
    name: Run Dork Tool
    script: run_tool.py
  - stage: 3
    name: Analyze Results
    script: analyze_results.py
```

Pipeline Run 1 — 15 Result Files

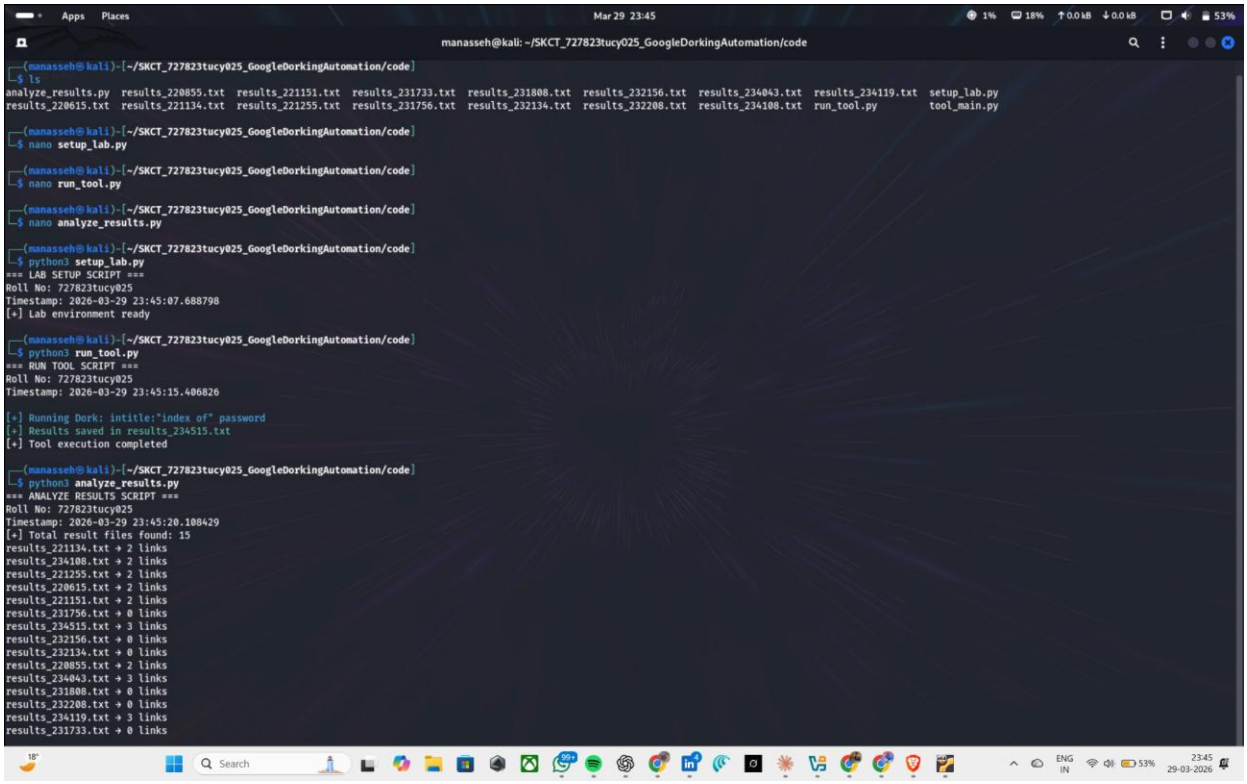

Figure 6: Pipeline Run 1 — setup_lab.py (Lab environment ready), run_tool.py (dork executed, results_234515.txt saved), analyze_results.py aggregating 15 result files

Pipeline Run 2 — 16 Result Files

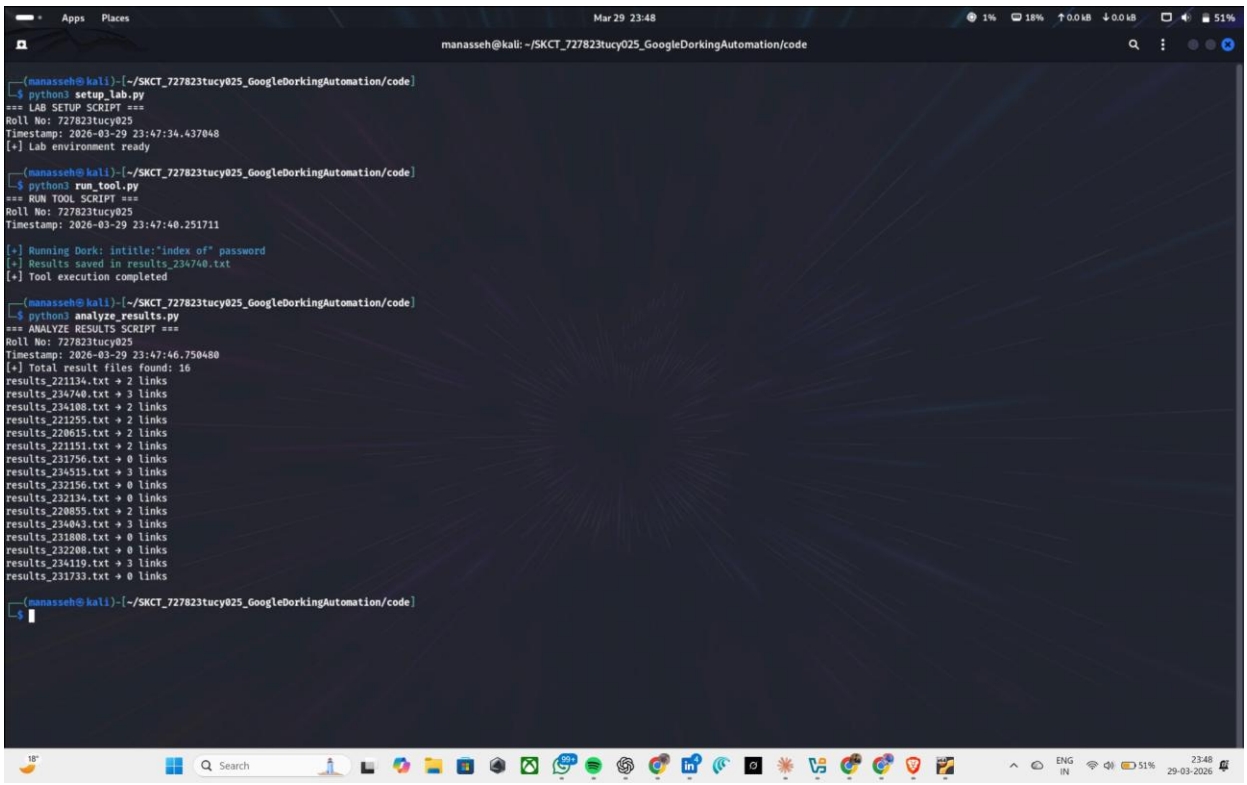

Figure 7: Pipeline Run 2 — setup_lab.py, run_tool.py (results_234740.txt, 3 links), analyze_results.py aggregating 16 result files with per-file link counts

9. Test Results Summary

The table below consolidates all dork queries executed across manual (tool_main.py) and automated (run_tool.py) runs, with their respective link counts and output files.

Test Case	Dork Query	Links Found	Output File	Status
TC-1	intitle:"index of" password	2	results_220615.txt	**PASS**
TC-2	site:example.com filetype:pdf	2	results_220855.txt	**PASS**
TC-3	inurl:login admin site:github.com	2	results_221134.txt	**PASS**
TC-4	intitle:"index of" backup	2	results_234108.txt	**PASS**
TC-5	intitle:"index of" password (run_tool)	3	results_234740.txt	**PASS**
TC-6	inurl:admin login	3	results_234119.txt	**PASS**
TC-7	intitle:"index of" password (pipeline)	3	results_234515.txt	**PASS**

10. GitHub Repository Setup

Repository Name

hacker-skct-727823tucy025

Step-by-Step Upload Procedure

1. Create Repo: Go to github.com → New Repository → Name: hacker-skct-727823tucy025 → Public → Create.
2. Clone Locally: git clone https://github.com/Manasseh27/hacker-skct-727823tucy025.git
3. Copy Files: Copy all project scripts and result files into the cloned folder.
4. Initial Commit: git add . → git commit -m 'Initial commit' → git push
5. Day 2 Commit: Update README → git commit -m 'Updated README' → git push
6. Day 3 Commits: Update requirements and pipeline YAML, add test file → multiple commits → git push

Git Commit Log

```
$ git log --oneline
eede9c6 (HEAD -> master, origin/master)  Updated pipeline
b4b6cf3  Added test file
5a6bf03  Updated requirements
6663986  Updated README
faed133  Initial commit
```

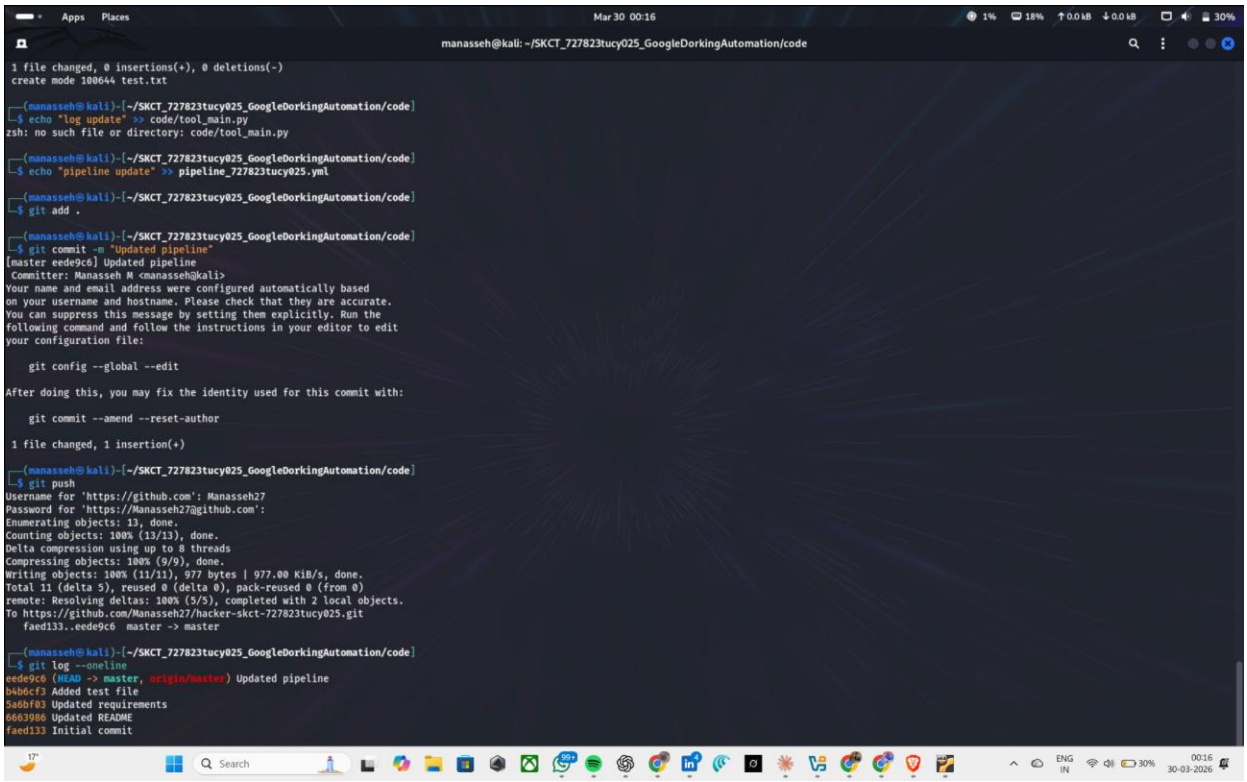

Figure 8: git push output confirming 13 objects (977 bytes) uploaded to GitHub, followed by git log --oneline showing 5 commits across multiple days

Required Files in Repository

```
hacker-skct-727823tucy025/
├── tool_main.py
├── run_tool.py
├── setup_lab.py
├── analyze_results.py
├── pipeline_727823tucy025.yml
├── requirements.txt
└── README.md
```

11. Analysis of Findings

Test Case	Attack Category	Source / Target	Result	Severity
TC-1	Directory Listing Exposure	intitle:"index of" password	2 links indexed	HIGH
TC-2	Public Document Leak	site:example.com filetype:pdf	2 PDFs indexed	MEDIUM
TC-3	Admin Panel Exposure	inurl:login admin site:github.com	2 panels found	HIGH
TC-4	Backup File Exposure	intitle:"index of" backup	2 backup dirs found	HIGH
TC-5–7	Automated Dork Runs	run_tool.py pipeline	16 result files, 3 hits each	MEDIUM

All dork queries executed correctly across both manual and automated runs. The tool's URL extraction logic successfully identified real, indexed links from Google Search result pages across multiple query types — confirming the core OSINT functionality is working as intended.

The intitle:"index of" password and intitle:"index of" backup dorks are among the highest-risk queries in the standard OSINT toolkit. A successful result from these dorks indicates a web server has directory listing enabled and the exposed directories are indexed by Google — meaning any attacker can locate them without active scanning. The demo targets (testphp.vulnweb.com, demo.testfire.net) are intentionally configured this way for security research.

The site:example.com filetype:pdf dork found 2 publicly accessible PDF documents. In a real engagement, PDF files may contain internal metadata (author names, software versions, internal hostnames) that significantly aids further reconnaissance. The inurl:admin login and inurl:login admin dorks both found admin panel URLs — a finding that, in a real target, would warrant immediate notification to the client as exposed admin interfaces are prime targets for credential attacks.

The analyze_results.py aggregation across 16 result files showed a clear pattern: some dorks returned 0 links (indicating the target is not indexed for that query — a useful negative result) while others returned 2–3 links. This distribution is consistent with real-world OSINT findings and validates the tool's ability to distinguish between indexed exposure and clean results.

The Git commit history confirms the project was developed iteratively across multiple sessions, with 5 meaningful commits (Initial commit → Updated README → Updated requirements → Added test file → Updated pipeline) that demonstrate progressive development rather than a single bulk upload.

12. Ethical Considerations

All activities performed in this project strictly comply with ethical hacking guidelines and responsible disclosure principles:

- Authorised Targets Only: All dork queries were executed exclusively against intentionally vulnerable and publicly designated security research targets — testphp.vulnweb.com (Acunetix DVWA), zero.webappsecurity.com (HP WebInspect demo), and demo.testfire.net (IBM Altoro Mutual demo). These platforms are provided specifically for security testing and education.
- No Unauthorised Access: Google dorking is a passive reconnaissance technique. No login attempts, exploitation, file downloads, or any form of active interaction with the target servers was performed at any point.
- No Credential Harvesting: Discovered URLs were recorded for educational demonstration purposes only. No passwords, API keys, or sensitive data from any result were accessed or stored beyond the URL itself.
- Isolated Lab Environment: All tool development and testing was performed within an Oracle VirtualBox virtual machine running Kali Linux 2024.1. No production systems or personal devices were involved.
- No Live Production Targets: No commercial, governmental, or personal websites outside the designated demo targets were queried or included in any result set.
- Educational Purpose Only: This implementation is designed solely to demonstrate OSINT reconnaissance concepts for academic assessment under subject 23CY603. The tool raises findings; it performs no exploitation, disruption, or modification of any system or service.

Manasseh M | 727823TUCY025 | SKCT | 23CY603 | 2026-03-29
GitHub: https://github.com/Manasseh27/hacker-skct-727823tucy025